

GO (GOLANG) - KEY TERMS EXPLAINED (For Absolute Beginners)

DATE : _____

TOPIC : Go - Key Concepts Cheat Sheet

CUES

What is this page? ☆

- Meanings of important Go terms.
- For someone who is new to coding.



How to use?

Whenever you see any term in Go code and don't understand it, come here and read it.

Think of it like...

Building a house. Each term is like a material or tool used to build the whole house (our program).



Remember



Don't try to memorize everything at once. Understand one by one while learning Go.

NOTES

This page explains the basic technical words/concepts used in Go (Golang) in very simple language with real-life examples.



Package

A package is like a folder that contains related Go code (functions, variables, types, etc). Every Go program must belong to a package.
Example: `package main`

`package main`

→ We are using the "main" package in our program.



Import

Import is used to use code written in another package. Go has many ready-made packages. We import them when we need their features.
Example: `import "fmt"`

`import "fmt"`

→ We are importing the "fmt" package to print text.



Library

A library is a collection of pre-written code that others can use. In Go, packages are also called libraries.
Example: `fmt`, `math`, `strings` are libraries.

`import "math"`

→ "math" is a library that has many math functions.



Program

A program is a set of instructions that the computer follows to do a task. Our Go code is a program.

Our whole Go file when run becomes a program.



main Package

The main package is special. It tells Go that this is the starting point of the program.
Example: `package main`

Only main package can be run directly.



main() Function

`main()` is a special function. Every Go program must have a `main()` function. This is where the program starts running.
Example:
`func main() { ... }`

`func main() {
 // program starts here
}`
→ Where execution begins.



Function

A function is a block of code that does a specific job. We can call it whenever we need.
Example:
`func add(a int, b int) int { ... }`

Helps in reusing code.



Variable

A variable is a container that stores data (like a box that holds a value). The value can change.
Example: `var name string = "Alice"`

Think of it like a box with a label.



Constant

A constant is a value that cannot change once set.
Example: `const PI = 3.14`

Like writing something with permanent ink.



Data Type

Data type tells Go what kind of data a variable can hold. Examples: `int` (whole number), `float64` (decimal number), `bool` (true/false), `string` (text)
Example: `var age int = 20`

Tells Go what kind of values are allowed.



Statement

A statement is an instruction that Go can execute.
Example: `fmt.Println("Hello")`

Each line of code (most lines) is a statement.



Expression

An expression is a combination of values, variables and operators that produces a result.
Example: `5 + 3` or `age > 18`

Evaluates to a single value (true, 8, "Hello", etc.)



Module (go.mod)

A module is a project. The `go.mod` file tells Go the module name and which packages it depends on.
Example: `module myproject`

Helps Go manage dependencies (libraries) in your project.



SUMMARY

- Package = Folder
- Import = Use code from another package
- Library = Collection of ready-made code (Packages are libraries)

- main package + `main()` function = Starting point
- Variable = Stores changeable data
- Constant = Value that never changes
- Data Type = Kind of data allowed

- Function = Reusable block of code
- Statement = Instruction
- Expression = Produces a value
- Module = Our project + dependencies

Key Idea:

All these small parts work together to build a Go program.



GO (GOLANG) FUNDAMENTALS – CORNELL NOTES

DATE: _____

TOPIC: Introduction to Go

CUES	NOTES
What is Go? Why was it created? ☆ Who created it?	1. INTRODUCTION TO GO - Go is a modern, open source programming language. - Created by Google in 2007 by Robert Griesemer, Rob Pike, and Ken Thompson. - It is simple, fast, safe, and concurrent. - Go is statically typed and compiled. - It is designed for efficiency and scalability. 
Hello World in Go How to write and run a simple Go program? </>	2. HELLO WORLD PROGRAM <pre>1 package main 2 3 import "fmt" 4 5 func main() { 6 fmt.Println("Hello, World!") 7 }</pre> How to Run: 1. Save the file as main.go 2. Open terminal 3. Run: go run main.go Output: Hello, World!
Go Program Structure What are the basic parts of a Go program? 	3. GO PROGRAM STRUCTURE <pre>1 package main // Every Go program starts with a package 2 3 import "fmt" // Import packages 4 5 func main() { // main() is the entry point 6 fmt.Println("Welcome to Go!") // Statement inside main 7 } // End of main function</pre> → package: Defines the package name. → import: Used to include packages. → func main(): Program execution starts here. → Statements: Instructions are written here.
Features of Go What makes Go unique and powerful? ⚡	4. FEATURES OF GO - Simple & Clean Syntax - Fast Compilation - Built-in Concurrency (Goroutines) - Garbage Collected - Strong Standard Library - Cross Platform - Statically Typed - Great for Scalable Systems 
Go Command Basics Important commands every beginner should know. >_	5. BASIC GO COMMANDS <pre>go version</pre> → Check Go version <pre>go run file.go</pre> → Run a Go program <pre>go build</pre> → Build the program <pre>go fmt file.go</pre> → Format the code <pre>go install</pre> → Install a package <pre>go mod init modname</pre> → Initialize module <pre>go mod tidy</pre> → Add missing & remove unused modules
Questions (Review) ?	6. REVIEW QUESTIONS - What is Go language? - Who developed Go language and why? - How do you print output in Go? - What is the entry point of every Go program? - What is the difference between go run and go build? 
LeetCode Practice (Easy) 	7. LEETCODE PRACTICE (EASY) 1. 258. Add Digits Given an integer num, repeatedly add all its digits until the result has only one digit, and return it. Example: Input: num = 38 Output: 2 Explanation: 3+8=11, 1+1=2 Link: https://leetcode.com/problems/add-digits/ 2. 1480. Running Sum of 1d Array Given an array nums. We define a running sum of an array as runningSum[i] = sum(nums[0]...nums[i]). Return the running sum of nums. Example: Input: nums = [1,2,3,4] Output: [1,3,6,10] Link: https://leetcode.com/problems/running-sum-of-1d-array/



SUMMARY

In this page, we learned about Go, why it was created, wrote our first program, understood its structure, explored key features, basic commands, and solved easy problems to strengthen our understanding.

GO (GOLANG) FUNDAMENTALS – CORNELL NOTES

DATE : _____
TOPIC : Basic Syntax & Data Types

CUES

What is Syntax?

Rules that decide how code should be written.



What are Data Types?

Different kinds of data Go can store.



Variables Recap

How to store data in a box.



Constants Recap

Fixed values that cannot change.



Type Conversion

Changing one type of data into another type.



Review Questions

Test your understanding!



NOTES

1. BASIC SYNTAX OVERVIEW

- Go is case-sensitive: "Name" and "name" are different.
- Every statement ends with a newline (press Enter).
- Code blocks are written inside { }.
- Comments help explain code (ignored by Go).
 - Single line comment: `// this is comment`
 - Multi line comment: `/* this is multi line comment */`

```
1 package main // package name
2
3 import "fmt" // import library
4
5 func main() { // start of main function
6     fmt.Println("Hello, Go!") // print text
7 } // end of main function
```

Output:
Hello, Go!

2. DATA TYPES IN GO (Explained Simply)

Data types tell Go what kind of data a variable can hold. Go has many types. Here are the most common ones.

Category	Type	What it means (Simple)	Example	Notes
Numbers (Whole)	int	Whole numbers (can be + or -)	var a int = -10	Most commonly used
	int8, int16, int32, int64	Whole numbers with specific size	var b int16 = 200	int8 (small) to int64 (very big)
	uint, uint8, uint16, uint32, uint64	Whole numbers (only +ve)	var c uint = 20	u in uint means unsigned (no negative)
Numbers (Decimal)	float32, float64	Decimal numbers (fractional part)	var d float64 = 3.14	float64 is default
Boolean	bool	Only two values: true or false	var e bool = true	Used in conditions
Text	string	Text (sequence of characters)	var f string = "Go"	Always use " " for strings
Others	byte	Special type for 0 to 255	var g byte = 65	Alias for uint8 (used for bytes)
	rune	Represents a character (Unicode)	var h rune = 'A'	Alias for int32



What is uint? Why it exists?

"uint" means unsigned integer. It can store only positive numbers (0, 1, 2, 3 ...). It is useful when we know our number will never be negative. It allows Go to store bigger positive values.

3. VARIABLES IN GO (Recap + Example)

- Use the keyword `var` to create a variable.
- Go automatically understands the type if we assign a value.
- Variables can change their value.

```
1 package main
2 import "fmt"
3
4 func main() {
5     var name string = "Alice"
6     age := 22 // short hand declaration
7     fmt.Println(name, age)
8 }
```

Output:
Alice 22

4. CONSTANTS IN GO

- Use `const` keyword.
- Value cannot change.
- Good for values that remain same.

```
1 const PI = 3.14159
2 const APP_NAME = "GoApp"
3
4 func main() {
5     fmt.Println(PI, APP_NAME)
6 }
7
```

Output:
3.14159 GoApp

5. TYPE CONVERSION

- You cannot directly assign one type to another.
- Use the type name to convert.
- Common example: `int ↔ float64`

```
1 package main
2 import "fmt"
3
4 func main() {
5     var a int = 10
6     var b float64 = float64(a)
7     fmt.Println(b)
8 }
```

Output:
10

- What is the rule about case sensitivity in Go?
- What is the difference between `int` and `uint`?
- Which data type should we use to store text?
- How do you create a variable in Go?
- What is the difference between `var` and `:=`?
- How do you convert an `int` value to `float64`?
- Can a constant value be changed? Why or why not?



7. LEETCODE PRACTICE (EASY) – Using only what we learned (variables, types, basic I/O, arithmetic)

1. 1929. Concatenation of Strings

Given two strings `firstName` and `lastName`, return the full name. The full name should be in the format "`firstName lastName`".

Example:

Input: `firstName = "John", lastName = "Doe"`

Output: "John Doe"

What we need to know: string, concatenation (+ operator)

Link: <https://leetcode.com/problems/concatenation-of-strings/>

2. 258. Add Digits

Given an integer `num`, repeatedly add all its digits until the result has only one digit, and return it.

Example:

Input: `num = 38`

Output: 2

Explanation: $3+8 = 11$, $1+1 = 2$

What we need to know: `int`, `%`, `/`, `+`, while loop (we'll learn loop later, but you can try with basic math for now)

Link: <https://leetcode.com/problems/add-digits/>



SUMMARY

- We learned basic syntax, different data types, variables, constants and type conversion in Go.
- These are the foundation concepts to build any Go program.
- Practice small problems daily to get comfortable!

Keep Practicing,
Keep Learning! 🚀

GO (GOLANG) FUNDAMENTALS – CORNELL NOTES

DATE : _____

TOPIC : Operators, Input/Output
& Basic Program Flow

CUES

What are Operators?

Symbols that perform operations on values.



NOTES

1. OPERATORS IN GO

Operators are special symbols that perform operations on values and variables.
Here are the most common ones:

Type	Operators	What it does	Example	Result
Arithmetic	+ - * / %	Basic math operations	10 + 3	13
			10 / 3	3 (int division)
			10 % 3	1 (remainder)
Relational (Comparison)	== != > < >= <=	Compare two values	5 >= 3	true
Logical	&& !	Combine conditions	true && false	false
Assignment	= += -= *= /= %=	Assign or update values	x += 5	x = x + 5

Example in Code:

```
1 package main
2 import "fmt"
3
4 func main() {
5     a := 10
6     b := 3
7     fmt.Println("a + b =", a+b)
8     fmt.Println("a - b =", a-b)
9     fmt.Println("a * b =", a*b)
10    fmt.Println("a / b =", a/b)
11    fmt.Println("a % b =", a%b)
12 }
```

Output:

```
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
```

Take Input from User

How does Go read data that a user types?



2. TAKING INPUT FROM USER

- We use the fmt package to read input.
- The Scan, Scanf, and Scanln functions read user input.

Example: Read a number from user

```
1 package main
2 import "fmt"
3
4 func main() {
5     var num int
6     fmt.Print("Enter a number: ")
7     fmt.Scanln(&num) // &num means store input in num
8     fmt.Println("You entered:", num)
9 }
```



Why & before variable?

Go needs to know where to store the input. & gives the memory address of the variable.

Sample Run:

```
Enter a number: 27
You entered: 27
```

Output to User

How to show results on the screen?



3. OUTPUT IN GO

- We already know fmt.Println() prints with a new line at the end.
- fmt.Print() prints without a new line.

```
1 package main
2 import "fmt"
3
4 func main() {
5     fmt.Print("Hello, ")
6     fmt.Scanln("Go!")
7     fmt.Println("Welcome!")
8 }
```

Output:

```
Hello, Go! Welcome!
```

Program Flow

How can our program make decisions or repeat things?



4. BASIC PROGRAM FLOW

a) IF - ELSE (Decision Making)

```
1 package main
2 import "fmt"
3
4 func main() {
5     var age int
6     fmt.Print("Enter your age: ")
7     fmt.Scanln(&age)
8     if age >= 18 {
9         fmt.Println("You are an adult.")
10    } else {
11        fmt.Println("You are a minor.")
12    }
13 }
```

Sample Run:

```
Enter your age: 16
You are a minor.
```

Note:

If condition is true, first block runs. Otherwise, else block runs.

b) FOR LOOP (Repeating)

```
1 package main
2 import "fmt"
3
4 func main() {
5     fmt.Println("Counting 1 to 5:")
6     for i := 1; i <= 5; i++ {
7         fmt.Println(i)
8     }
9 }
```

Sample Run:

```
Counting 1 to 5:
1
2
3
4
5
```



Note:

Loop runs until the condition (i <= 5) is no longer true.

LeetCode Practice (Easy)

Practice small problems using what we learned!



5. REVIEW QUESTIONS

- What are arithmetic operators in Go? Give examples.
- What is the difference between == and = ?
- How do you take input from a user in Go?
- What is the use of & when taking input?
- How do you print output in Go without a new line?
- What is the difference between if and if-else?
- How does a for loop work? Give its basic syntax.



6. LEETCODE PRACTICE (EASY) – Based on what we learned (operators, input/output, if, loop)

1. 101. Symmetric Number

Given an integer x, return true if x is a palindrome, and false otherwise.
An integer is a palindrome when it reads the same backward as forward.
(Do not convert the integer to string.)

Example:

Input: x = 121

Output: true

Explanation: 121 reads the same backward and forward.

What we need to know: %, /, while/for loop, comparison.

Link: <https://leetcode.com/problems/symmetric-number/>



2. 268. Missing Number

Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.

Example:

Input: nums = [0,1,3]

Output: 2

Explanation: n = 3 because numbers are in range [0,3].

2 is the missing number.

What we need to know: for loop, sum formula or logic, basic math.

Link: <https://leetcode.com/problems/missing-number/>



SUMMARY

- We learned: Operators, Input/Output, If-Else, For Loop in Go.
- These are building blocks to create powerful programs.
- Practice small programs every day to get better!

Tip:

Write code, run, make mistakes, fix them, and learn!



Keep Practicing,
Keep Growing!